NATURAL LANGUAGE PROCESSING

Mining the Visitor Experience

Sentiment, Topics, Explainability and Retrieval-Augmented Generation from Rome Colosseum Reviews

ITC 6110 - Natural Language Processing

Final Group Project | Spring Semester 2026

CORPUS	BEST MACRO-F1	PIPELINE
8,285 reviews	**0.47 | XGBoost**	**NLP to RAG**

Prepared from the executed ITC6110 project notebook and recorded outputs

Contents

Abstract

This project develops an end-to-end natural language processing pipeline for 8,285 English-language visitor reviews of the Rome Colosseum. The work connects data quality controls, linguistic normalization, exploratory visualization, sparse and dense feature engineering, supervised sentiment classification, explainability, topic modelling, retrieval-augmented generation, and a prototype user interface. The dataset is strongly imbalanced: 7,472 reviews are labelled Positive, 453 Neutral, and 360 Negative. Missing values occur only in travel month and travel season, with 19 observations in each field, and no duplicate rows are detected. Text is normalized through lowercasing, URL and username replacement, stop-word removal with negations preserved, emoji and emoticon removal, non-alphabetic character handling, repeated-letter normalization, short-word filtering, lemmatization, and tokenization.

Three numerical representations are investigated. Bag of Words and TF-IDF produce 2,654 training features after document-frequency filtering, while Word2Vec learns 100-dimensional vectors over a vocabulary of 3,406 tokens. Word2Vec supports semantic-neighbour queries and document embeddings; a t-SNE projection of 2,000 document vectors is used to examine review-length groups. For supervised learning, TF-IDF feeds Logistic Regression and XGBoost, while a compact three-epoch bidirectional LSTM uses token sequences. XGBoost achieves the strongest recorded test results, with 0.9113 accuracy and 0.47 macro-F1. Logistic Regression obtains 0.9065 accuracy and 0.41 macro-F1, while the BiLSTM reaches approximately 0.90 accuracy but only 0.3343 macro-F1. These results show that headline accuracy is dominated by the Positive class and must be interpreted alongside class-specific recall and macro-F1.

Global SHAP analysis is applied to the Logistic Regression model. A ten-topic LDA model obtains a coherence score of 0.3926 and log-perplexity of -7.3292, indicating interpretable but overlapping visitor themes. The RAG prototype uses all-MiniLM-L6-v2 embeddings, cosine retrieval over cleaned reviews, and DistilGPT2 generation. Retrieval scores are moderate, but one generated response is repetitive and unsupported, exposing a grounding failure. The Streamlit file demonstrates the intended interaction pattern but remains disconnected from the retriever and generator, and no deployment evidence is recorded. The project therefore establishes a broad working NLP pipeline while identifying class imbalance, token-cleaning artefacts, RAG faithfulness, and deployment integration as the most important areas for improvement.

1. Introduction

Online visitor reviews contain operational, emotional, and experiential information that is valuable to travellers, destination managers, and cultural-heritage organisations. A single review may describe historical interest, crowding, queues, ticketing, tour quality, heat, accessibility, or perceived value. Across thousands of reviews, these observations form a noisy but useful record of visitor experience. Natural language processing provides a systematic way to transform that record into quantitative features, predictive models, latent topics, and question-answering interfaces.

The objective of this project is to demonstrate practical use of techniques across the NLP pipeline rather than to optimize a single model in isolation. The work therefore begins with a labelled dataset suitable for sentiment classification and proceeds through cleaning, normalization, tokenization, vectorization, visualization, modelling, explanation, retrieval, generation, and interface design. This breadth is important because downstream results depend on upstream decisions. For example, removing every short word may reduce vocabulary size, but it can also erase sentiment-bearing negations. Similarly, a classifier can report high accuracy on an imbalanced dataset even when it fails to recognise minority classes.

Four research questions organise the analysis. First, what data-quality and normalization decisions are appropriate for Colosseum reviews? Second, how well do sparse lexical features and learned word embeddings represent the corpus? Third, how effectively do traditional machine-learning and deep-learning models classify sentiment under severe imbalance? Fourth, can the same corpus support interpretable topic discovery and a grounded retrieval-augmented conversational prototype?

The notebook records completed experiments and their outputs. This report uses those recorded values as the evidential boundary: it does not claim tuning, deployment, or evaluation that the notebook does not demonstrate. It also resolves two documentation inconsistencies. The notebook's old Findings comment lists accuracies of 0.7517 and 0.7353, but the executed outputs show 0.9065 for Logistic Regression and 0.9113 for XGBoost. In addition, the t-SNE code labels the grouping as “Trip Type” although the values come from review_length_tier; the figure is therefore interpreted as review-length grouping.

2. Data Collection and Understanding

The project uses the “Rome Colosseum Visitor Reviews 2019-2026” dataset distributed through Kaggle. The notebook describes 8,285 verified visitor reviews published between May 2019 and June 2026, with travel periods from June 2018 to June 2026. The CSV contains 15 variables: publication date, travel month, publication platform, title, review text, rating, trip type, helpful votes, word count, review-length

tier, sentiment label, publication year, publication month, travel season, and verified-travel status. Review text is English and the dataset already includes labels required for supervised sentiment analysis.

Initial inspection confirms a rectangular dataset of 8,285 rows and 15 columns. Five variables are stored as integers, nine as strings, and is_verified_travel as a Boolean. The publication platform has two observed values, OTHER and MOBILE. The analysis eventually retains review text, sentiment label, numerical rating, and review-length tier for the primary NLP workflow. The additional fields remain useful for exploratory grouping and for auditing the meaning of labels.

The sentiment distribution is highly asymmetric. Positive accounts for 7,472 reviews (90.19%), Neutral for 453 (5.47%), and Negative for 360 (4.35%). This imbalance reflects a plausible visitor-review environment, but it fundamentally changes model interpretation. A trivial classifier that always predicts Positive would already achieve about 90.2% accuracy. Accuracy near 0.91 is therefore not evidence of balanced sentiment recognition. The most informative measures are minority-class recall, macro-F1, the confusion matrix, and comparison against the majority-class baseline.

The dataset is appropriate for the project because it provides long-form and short-form text, labelled sentiment, sufficient volume for classical and neural experiments, and recurring themes that can support topic modelling and retrieval. It also has real operational questions: visitors frequently discuss advance booking, queue management, guided tours, the arena floor, the underground area, Palatine Hill, crowding, and heat. These subjects allow the same corpus to be reused for classification, semantic similarity, topic analysis, and question answering.

There are nevertheless limitations. The sentiment labels are supplied by the dataset rather than independently annotated in the notebook, so label quality and the relationship between rating and sentiment are not audited. Reviews may also be affected by platform-specific selection bias: visitors who post are not a random sample of all visitors. The time span crosses major tourism disruptions and policy changes, but the notebook does not perform temporal stratification. Finally, the data source description says reviews are verified, while the report cannot independently validate collection or licensing beyond the provided dataset metadata.

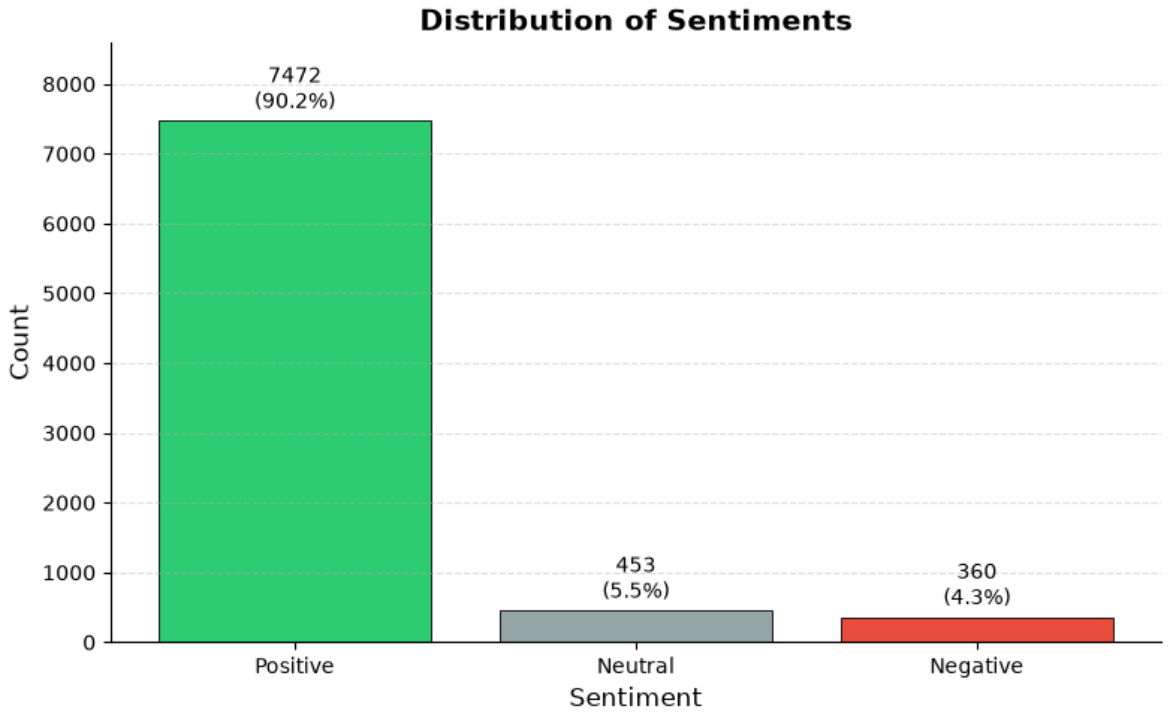

Figure 1. Sentiment class distribution in the full dataset. The dominance of Positive reviews motivates stratified splitting and macro-averaged evaluation.

3. Data Preprocessing and Normalization

3.1 Structural quality checks

The notebook first checks dimensionality, duplicates, and missingness. Duplicate removal does not change the row count: both before and after the operation, the dataset contains 8,285 rows. Missing values are limited to 19 entries in travel_month and 19 in travel_season; all other columns are complete. Both incomplete categorical fields are filled with their respective modal value, after which a second missingness check returns zero null values throughout the dataset. Mode imputation preserves the full corpus and is proportionate because only 0.23% of rows are affected in each field. However, these two variables do not drive the primary sentiment models, so the imputation has little direct effect on classification.

The workflow then selects text, sentiment_label, rating, and review_length_tier. This reduction focuses computation and clarifies the modelling target. The sentiment label is preserved separately before text transformation. Review-length tier is later used for visual grouping, although the relevant t-SNE cell inconsistently calls it trip type. Index resetting before the train-test split is a useful safeguard because tokenized and untokenized series must remain aligned with labels.

3.2 Linguistic normalization

The first normalization step converts every review to lowercase, reducing duplicate vocabulary caused only by capitalization. URLs are detected with a regular expression and replaced by the token URL. Replacement is preferable to silent deletion when the presence of a link may itself be meaningful, although later cleaning may alter the uppercase token. Email addresses and genuine @username patterns are replaced by USER, which reduces direct identifiers and prevents highly specific handles from inflating the vocabulary.

English stop words are removed using NLTK, with no, not, nor, and never deliberately retained. Preserving negation is especially important for sentiment: “worth” and “not worth” should not collapse to the same representation. The code then removes Unicode emojis and a set of ASCII emoticons. This simplifies tokenization, but it also discards potentially useful affective signals. A stronger future design would translate common emojis into semantic tokens such as EMO_POSITIVE or EMO_NEGATIVE rather than remove them.

Non-alphabetic material is normalized with an option to retain ., !, and ?. This choice attempts to keep expressive punctuation while removing noisy symbols and digits. Consecutive letters are capped at two repetitions, turning exaggerated forms such as “soooo” into “soo.” The approach reduces sparsity but does not necessarily recover the canonical dictionary form “so.” Short words below three characters are removed, with no and not preserved. This is consistent with the sentiment objective, yet contractions and domain abbreviations require care. The recorded Word2Vec neighbours still contain tokens such as tour. and colosseum., showing that punctuation remained attached to some words. That artefact splits semantic equivalents across separate vocabulary entries.

Lemmatization uses NLTK WordNet with part-of-speech information. Compared with stemming, lemmatization aims to return valid base forms and is therefore more interpretable in word clouds, similarity lists, SHAP output, and topic keywords. The process is computationally heavier, but the corpus size is manageable. Finally, whitespace tokenization produces a separate series of token lists while retaining cleaned strings for sparse vectorizers.

The preprocessing sequence is broad and addresses most required categories: missing data, duplicates, case, URLs, identifiers, stop words, emojis, emoticons, special characters, elongated words, short words, lemmatization, and tokenization. Its principal weakness is order sensitivity. Stop-word removal occurs before lemmatization, punctuation is partly retained, and whitespace tokenization allows punctuation-bearing variants to survive. A production pipeline should use a single tested tokenizer and add unit tests for negation, email replacement, apostrophes, accented text, repeated characters, and empty reviews. It should also record how many documents become empty or unusually short after cleaning.

Figure 2. Word cloud for Positive reviews after normalization. Large terms summarize recurring favourable visitor language but should not be interpreted as statistical effect sizes.

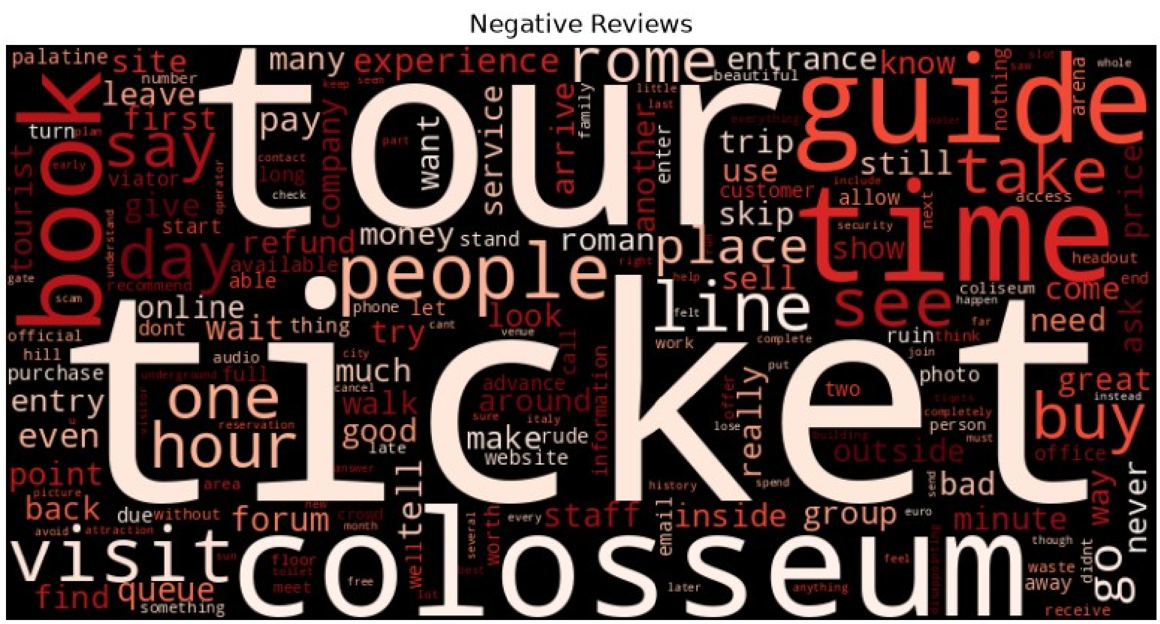

Figure 3. Word cloud for Negative reviews after normalization. The contrast with the Positive cloud highlights operational concerns but is sensitive to class size and preprocessing.

4. Feature Engineering and Text Visualization

4.1 Train-test design

The cleaned strings and encoded labels are split into training and test sets using an 80:20 ratio, random_state=42, and stratification. The resulting sets contain 6,628 training reviews and 1,657 test reviews. The encoded mapping is Negative = 0, Neutral = 1, and Positive = 2. Stratification successfully preserves the distribution: Positive represents 90.1931% of training instances and 90.1629% of test instances; the corresponding Negative proportions are both 4.3452%, while Neutral is approximately 5.46%. Fitting vectorizers and Word2Vec only on training data limits information leakage.

4.2 Bag of Words and TF-IDF

Bag of Words uses CountVectorizer with a maximum of 5,000 features, minimum document frequency of five, maximum document frequency of 0.8, and English stop-word removal. After filtering, the training matrix is 6,628 by 2,654 and the test matrix is 1,657 by 2,654. The representation is transparent: each column corresponds to a term and each value to its count. It is suitable for interpretable linear baselines but ignores word order and treats semantically related forms as independent dimensions.

TF-IDF uses matching frequency thresholds and vocabulary limits. Unlike raw counts, it downweights terms that occur broadly across reviews and emphasizes terms that are comparatively distinctive within a document. TF-IDF is selected for Logistic Regression and XGBoost because it combines computational efficiency with strong performance on many medium-sized text-classification tasks. It also works naturally with linear SHAP explanations because individual features correspond to human-readable terms. The notebook displays a sample matrix but does not report the complete TF-IDF matrix shape; because the vectorizer uses equivalent thresholds on the same training corpus, the visible sample also contains 2,654 columns.

Using both Bag of Words and TF-IDF demonstrates two sparse representations, but only TF-IDF is evaluated in downstream classifiers. A direct controlled comparison would strengthen the embedding section. It could hold the classifier, split, and hyperparameters constant and compare accuracy, macro-F1, training time, vocabulary size, and calibration. N-grams would be especially useful for negated phrases such as “not worth,” “no queue,” and “highly recommend,” which unigram representations cannot preserve directly.

4.3 Word2Vec and semantic similarity

Word2Vec is trained only on tokenized training reviews using 100 dimensions, context window five, minimum count five, ten epochs, four workers, and a fixed seed. The model learns a vocabulary of 3,406

tokens. Its most frequent vocabulary includes domain-relevant terms such as tour, colosseum, ticket, guide, visit, rome, history, book, queue, and recommend. Unlike sparse counts, Word2Vec represents each token as a dense vector whose position is shaped by surrounding words.

The required nearest-neighbour function is demonstrated with multiple queries. For tour, the strongest neighbours include tour. (0.9192), tours. (0.8064), private (0.7825), hire (0.7258), audio (0.7133), and knowledgeable. (0.7072). These relationships reflect forms and concepts associated with guided visits. For colosseum, neighbours include coliseum (0.9077), colosseum. (0.8468), colosseo (0.7400), forums. (0.7224), palatine. (0.7174), and the misspelling colloseum (0.7170). The embedding therefore captures spelling variants and nearby heritage sites, but the punctuation variants reveal incomplete normalization.

The query colloseum gives very high similarities to piazza, walkable, east, pantheon, and other location-related terms. Because the misspelling is less frequent, these neighbours may reflect a narrow set of travel narratives rather than a robust synonym cluster. Similarity values should therefore be interpreted in relation to token frequency and stability across random seeds.

Document vectors are formed by averaging the Word2Vec vectors for in-vocabulary tokens in each review. Reviews without known tokens receive a zero vector. This simple aggregation produces a 6,628 by 100 training matrix. Averaging is efficient and order-invariant, but it treats every token equally and loses syntax, negation scope, and sentence structure. TF-IDF-weighted averaging or Sentence-BERT embeddings would provide stronger document representations.

4.4 t-SNE visualization

A random sample of 2,000 document embeddings is projected to two dimensions using t-SNE with cosine distance, perplexity 30, random initialization, automatic learning rate, and random_state=42. The sample contains 1,159 Medium, 445 Long, 322 Short, and 74 Very Long reviews. The scatterplot colours points by review-length tier, despite its title referring to trip type.

The plotted points are broadly intermingled rather than forming clean, isolated length clusters. This is expected: an averaged semantic vector is intended to capture content, whereas review length is a coarse metadata attribute. Some local variation may occur because longer reviews combine more themes and average more tokens, but t-SNE preserves neighbourhoods rather than global distances. Apparent gaps, shapes, or cluster sizes should not be treated as formal evidence without quantitative validation. A stronger analysis would colour the same coordinates by sentiment, compute neighbourhood purity, compare PCA and UMAP, and repeat t-SNE across seeds.

Figure 4. t-SNE projection of 2,000 averaged Word2Vec document embeddings, correctly interpreted by review-length tier rather than trip type.

5. Supervised Sentiment Classification

5.1 Logistic Regression

The traditional linear baseline is multinomial Logistic Regression with the LBFGS solver, a maximum of 1,000 iterations, and random_state=42. It is fitted to training TF-IDF vectors and evaluated on the untouched test set. Recorded accuracy is 0.9065. Positive sentiment achieves precision 0.91, recall 1.00, and F1 0.95 over 1,494 examples. Negative sentiment achieves precision 0.75 but recall only 0.17, producing F1 0.27 over 72 examples. Neutral sentiment has zero precision, recall, and F1 over 91 examples. Macro-F1 is 0.41 and weighted F1 is 0.87.

The confusion matrix confirms that the model predicts the majority class for nearly all test reviews. Its accuracy is only about 0.45 percentage points above an always-Positive baseline. Logistic Regression is therefore useful as an interpretable reference but does not provide satisfactory three-class discrimination. Its relatively high Negative precision indicates that predicted Negative cases are often correct, yet the very low recall means most actual Negative reviews are missed.

5.2 XGBoost

XGBoost is trained on the same TF-IDF split with the multi-class log-loss evaluation metric and otherwise default notebook settings. It obtains 0.9113 accuracy, 0.47 macro-F1, and 0.88 weighted F1. Negative precision, recall, and F1 are 0.67, 0.25, and 0.36. Neutral scores are 0.62, 0.05, and 0.10. Positive scores are 0.92, 1.00, and 0.95.

Relative to Logistic Regression, XGBoost improves overall accuracy by 0.48 percentage points and macro-F1 by six percentage points. More importantly, it retrieves a larger share of Negative reviews and makes some correct Neutral predictions. Even so, 5% Neutral recall is operationally weak, and most minority observations remain classified as Positive. The XGBoost confusion-matrix code mistakenly titles the plot “Confusion Matrix for Logistic Regression,” but the matrix itself is calculated from XGBoost predictions; the report uses a corrected caption.

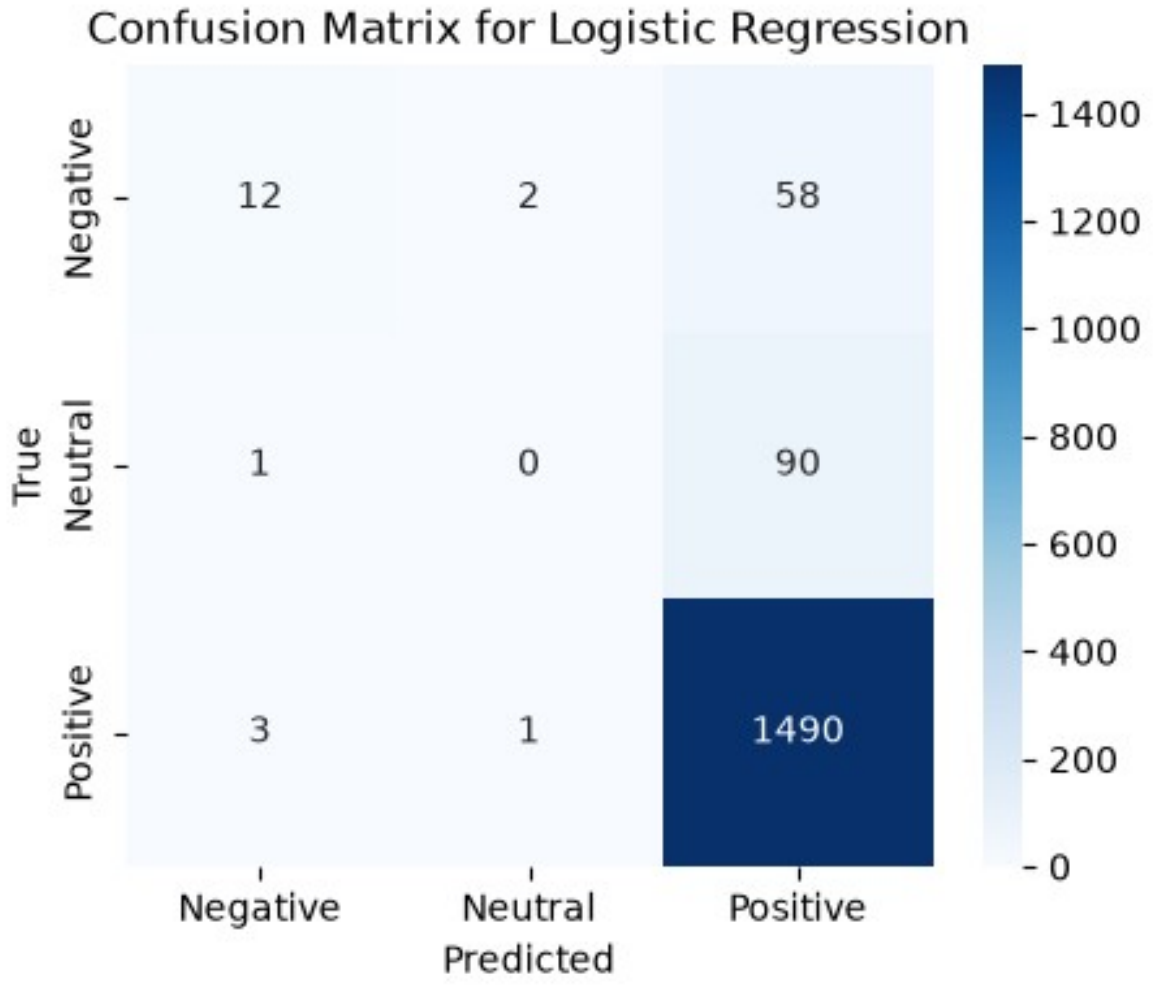

Figure 5. Confusion matrix for Logistic Regression on the stratified test set.

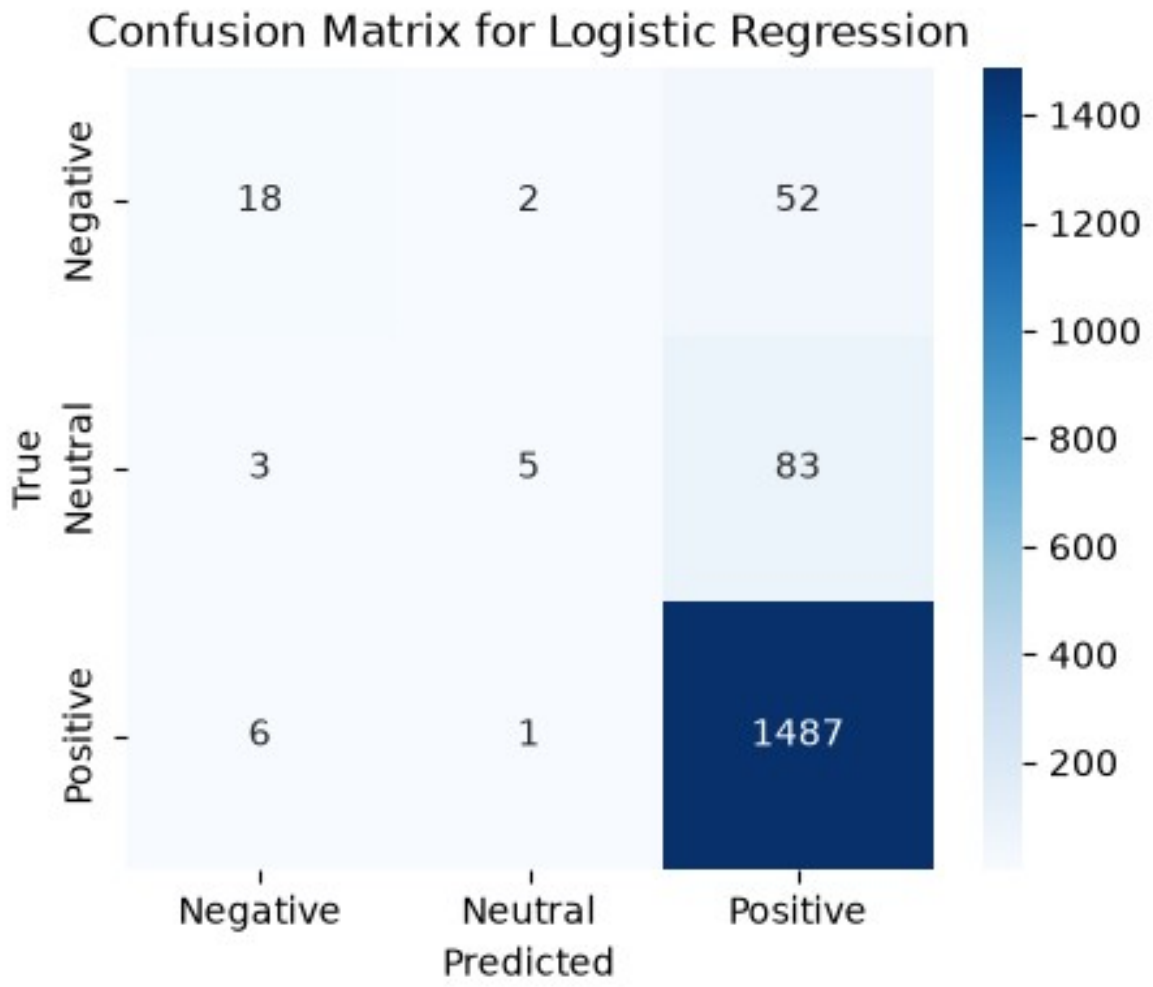

Figure 6. Confusion matrix for XGBoost on the stratified test set. The notebook's plot title is stale, but the displayed matrix uses XGBoost predictions.

5.3 Bidirectional LSTM

The deep-learning baseline is a compact PyTorch bidirectional LSTM. A vocabulary is constructed from training tokens, reserving indices for padding and unknown words and limiting the learned vocabulary to the 19,998 most common items. Each review is truncated or padded to 120 tokens. The model uses a 64-dimensional embedding layer, a 64-unit bidirectional LSTM, and a linear layer mapping the concatenated forward and backward hidden states to three classes. It is optimized with Adam at a learning rate of 0.001, unweighted cross-entropy, batch size 64, and three epochs.

The BiLSTM records approximately 0.90 accuracy and macro-F1 of 0.3343. Positive precision, recall, and F1 are 0.90, 1.00, and 0.95. Negative precision is 1.00, but recall is only 0.03 and F1 0.05. Neutral performance is zero across all three measures. The apparent Negative precision is based on very few predicted examples and should not be interpreted as strong minority modelling.

The neural model underperforms both traditional models on macro-F1. This does not show that deep learning is intrinsically unsuitable; it shows that this compact configuration, short training schedule, unweighted loss, and imbalanced data are insufficient. The embedding layer starts from random weights, and three epochs may not allow rare-class patterns to be learned. Class-weighted loss, focal loss, balanced sampling, pretrained embeddings, early stopping, validation-based tuning, and a transformer encoder are reasonable next steps.

5.4 Comparative interpretation

The most defensible ranking is XGBoost first, Logistic Regression second, and BiLSTM third, with macro-F1 as the main criterion. XGBoost offers the best minority-class recovery while retaining high overall performance. Logistic Regression remains attractive for speed and explanation. The BiLSTM demonstrates the required neural architecture but largely collapses to the majority class.

Model	Accuracy	Macro-F1	Negative recall	Neutral recall	Positive recall
Logistic Regression	0.9065	0.41	0.17	0.00	1.00
XGBoost	0.9113	0.47	0.25	0.05	1.00
BiLSTM	approx. 0.90	0.3343	0.03	0.00	1.00

The comparison also reveals an evaluation-design challenge. No model is tested against class weighting, resampling, threshold adjustment, or a calibrated majority baseline. There is no separate validation set or cross-validated hyperparameter search. A future experiment should optimize macro-F1 under nested or repeated stratified validation, report balanced accuracy and per-class precision-recall curves, and evaluate confidence calibration. Error analysis should sample false Positive, false Neutral, and false Negative cases to determine whether ambiguity, sarcasm, mixed sentiment, or label noise drives the failures.

6. Explainable AI with SHAP

Global explanation is applied to the Logistic Regression baseline with shap.LinearExplainer. SHAP values are calculated for the first 100 TF-IDF test examples, and a summary plot ranks features by their contribution across classes and observations. The notebook reports that the background training set contains 6,628 samples and that SHAP subsamples it to 100 background observations. This makes the calculation manageable but means the plot is an exploratory sample rather than a complete population explanation.

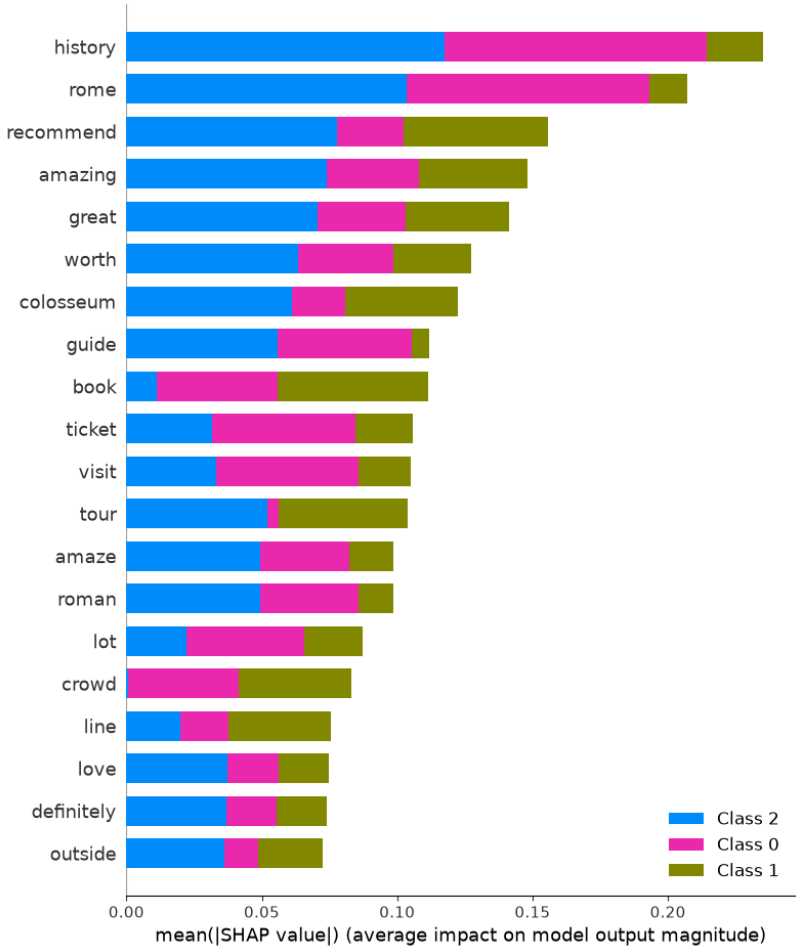

Figure 7. SHAP summary for the Logistic Regression TF-IDF classifier, based on 100 test observations and a subsampled background.

The plot connects individual lexical features to model outputs, which is particularly useful for checking whether the classifier relies on plausible sentiment terms or on spurious proxies. Features with large absolute SHAP values exert stronger local influence, while colour indicates the observed feature magnitude. Because this is a multiclass linear model and only 100 test rows are explained, interpretation should be class-specific and cautious. Correlated words may share attribution, and preprocessing artefacts can create separate features for punctuated variants.

SHAP explains model behaviour, not causal visitor experience. A token can be predictive because it correlates with a label in this dataset, not because it causes sentiment. The strongest validation would combine the global summary with local force or waterfall plots for correctly and incorrectly classified minority reviews. This would reveal whether missing negation phrases, mixed opinions, or vocabulary sparsity explains false predictions. Comparing SHAP patterns across Logistic Regression and XGBoost could also show whether non-linear interactions improve minority recognition.

7. Unsupervised Topic Modelling

The unsupervised component uses Gensim Latent Dirichlet Allocation. A dictionary is created from all tokenized reviews, initially containing 16,886 tokens. Terms occurring in fewer than ten documents or more than 10% of documents are removed, leaving 2,366 dictionary terms. The document-term corpus is then constructed and a ten-topic LDA model is trained with two passes, symmetric alpha, automatically learned eta, and random_state=42.

The recorded coherence score is 0.3926 and log-perplexity is -7.3292. Coherence indicates modest semantic consistency rather than sharply separated themes. This is credible for attraction reviews, where most documents combine several recurring elements. Perplexity is useful mainly for comparison with alternative models trained on the same preprocessing and corpus; the isolated value should not be treated as an intuitive quality percentage.

The visible topic keywords suggest operational and experiential themes. Topic 1 begins with sell, try, and staff, consistent with ticket sellers or visitor assistance. Topic 2 includes year, ancient, and gladiator, reflecting historical imagination. Topic 3 begins with arena, floor, and access. Topic 4 includes coliseum, even, and water, possibly mixing spelling variants with practical advice. Topic 5 includes highly, despite, and love, suggesting recommendation language. Topic 6 includes entry, amaze, and variants of "amazing." Topic 7 begins with monument, ancient, and use; Topic 8 with include, underground, and palatine; Topic 9 with beautiful, nice, and need; and Topic 10 with palatine, advance, and headout. These are provisional labels because the stored dataframe display truncates the remaining keywords.

Topic	Visible leading keywords	Provisional interpretation
1	sell, try, staff	Vendors, ticketing, and staff interaction
2	year, ancient, gladiator	History and Roman-era imagination
3	arena, floor, access	Arena-floor access and visit options
4	coliseum, even, water	Practical visit conditions and variants
5	highly, despite, love	Recommendation and positive appraisal
6	entry, amaze, amazing	Entry experience and emotional response
7	monument, ancient, use	Heritage significance and practical use
8	include, underground, palatine	Combined ticket areas and inclusions
9	beautiful, nice, need	Aesthetic appraisal and visitor needs
10	palatine, advance, headout	Advance booking and adjacent attractions

The topics overlap because the corpus concerns one attraction and preprocessing retains related spelling variants. Two passes are also a minimal training schedule. A stronger evaluation would vary topic count, passes, alpha, eta, and frequency thresholds; select models using coherence plus human interpretability; and report topic prevalence over time, rating, or sentiment. pyLDAvis is generated in

the notebook, enabling interactive inspection of topic separation and salient terms, but the HTML output is not a static figure that can be reproduced faithfully in this Word report.

8. Retrieval-Augmented Generation

The RAG prototype reuses cleaned reviews as retrievable documents. SentenceTransformer all-MiniLM-L6-v2 encodes every review into a normalized dense vector. A question is encoded with the same model, cosine similarities are computed, and the five highest-scoring reviews are concatenated into context. The prompt instructs the generator to use only the review context and to state when the evidence is insufficient. DistilGPT2 then generates up to 120 new tokens without sampling.

The retrieval stage demonstrates meaningful semantic matching. For the question “What advice do visitors give about queues and tickets?”, the top five cosine similarities range from approximately 0.656 to 0.695. Retrieved reviews discuss pre-booked tickets, separate queues, long waits, skip-the-line offers, online purchasing, toilet queues, and ticket touts. The evidence is therefore relevant to the question.

Generation is much weaker. The recorded answer repeatedly states, “If you are a fan of the game, you can always ask for a ticket,” despite context about Colosseum queues. The text is repetitive, semantically misplaced, and not grounded in the retrieved passages. This result shows that retrieval relevance does not guarantee answer faithfulness. DistilGPT2 is a small general causal language model rather than an instruction-tuned question-answering model, and the prompt plus concatenated context is insufficient to constrain it.

Two evaluation questions are recorded. “How can visitors avoid queues?” obtains a top retrieval score of 0.653728 and keyword coverage of 0.5 for the expected terms ticket and online. “What do reviews say about tours?” obtains 0.680912 and keyword coverage of 1.0 for tour and guide. These metrics are simple and useful for a smoke test, but two questions cannot establish quality. Keyword coverage can also reward an answer that mentions expected words while remaining factually wrong.

A credible evaluation should use a larger question set covering factual advice, synthesis, ambiguity, and unsupported requests. Human raters or an explicit rubric should score retrieval relevance, answer correctness, faithfulness to cited reviews, completeness, fluency, and refusal when evidence is absent. Automated measures could include recall@k on manually labelled relevant reviews, mean reciprocal rank, semantic answer similarity, and citation precision. Each answer should show its source excerpts and similarity scores. Replacing DistilGPT2 with an instruction-tuned model, using a structured prompt, limiting context to non-redundant passages, and post-checking every claim against retrieved text would address the observed failure.

9. Streamlit Interface and Deployment Status

The app.py file defines a simple Streamlit page titled “Colosseum Review Assistant.” It provides explanatory text, a text input, an Ask button, an Answer heading, and an expandable area intended for retrieved reviews. These components represent an appropriate minimum interaction pattern for a RAG demonstration: the user asks a natural-language question, reads an answer, and can inspect supporting evidence.

However, the current interface is a template rather than an integrated application. When the button is pressed, it explicitly displays “Connect the saved retriever and generator from the RAG cell here.” The answer and retrieved-review panels contain static instructions rather than calls to rag_answer. The notebook also prints deployment instructions but does not record a Hugging Face Space URL, build log, screenshot, or remote test. The requirement to deploy and evaluate user experience is therefore not evidenced by the available artifacts.

Integration requires moving preprocessing, embedding loading, retrieval, prompt construction, and generation into importable functions or a serializable service. The document embeddings should be computed once and cached rather than recomputed for every Streamlit rerun. The interface should display progress, validate empty input, handle model-loading errors, show cited reviews, and allow feedback. Secrets and model tokens should be stored through the deployment platform rather than hard-coded. A lightweight evaluation could ask representative users to rate answer usefulness, clarity, trust, latency, and ease of source inspection.

10. Challenges, Limitations, and Reproducibility

The central modelling challenge is severe class imbalance. All three classifiers learn the Positive class very well while performing poorly on Neutral and Negative reviews. Macro-F1 exposes this pattern; accuracy hides it. The dataset also contains semantic overlap: a review can praise the monument while criticising queues, making a single document label potentially coarse. Without label-audit samples, it is unclear how consistently mixed sentiment is handled.

Preprocessing reduces noise but also introduces artefacts. Punctuation-bearing tokens remain in Word2Vec, and removing emojis or short words may discard information. The order of stop-word filtering, lemmatization, and punctuation handling is not experimentally compared. Word clouds are qualitative and affected by corpus size. t-SNE is stochastic and its global geometry is not directly interpretable. LDA uses only two passes and ten topics without a documented model-selection study.

The supervised experiments use one stratified split, no separate validation set, and little or no hyperparameter tuning. This is acceptable for baseline comparison but insufficient for a final model-selection claim. Random seeds are set in many stages, yet Word2Vec uses multiple workers and PyTorch does not visibly set all deterministic seeds. Exact replication may therefore vary slightly. The Logistic Regression code also includes an n_jobs parameter that the installed scikit-learn version warns has no effect.

The RAG prototype presents the sharpest gap between component success and end-to-end quality. Dense retrieval finds relevant reviews, but generation produces an unsupported, repetitive response. The Streamlit layer is not connected and deployment is undocumented. These are not minor presentation issues; they are functional limitations that should be resolved before describing the chatbot as complete.

Reproducibility is supported by the notebook, local CSV, requirements.txt, fixed split seed, and recorded outputs. It would be improved by pinning all dependency versions, separating environment setup from analysis, saving trained artifacts, providing a single run script, logging package versions and hardware, and adding tests for preprocessing and retrieval. The notebook should also remove stale comments and correct figure labels so narrative conclusions cannot drift from executed evidence.

11. Future Work

The first priority is imbalance-aware classification. Class-weighted Logistic Regression, scale_pos_weight or sample weights for XGBoost, balanced minibatches, and focal loss for the BiLSTM should be compared under repeated stratified validation. Macro-F1, balanced accuracy, per-class average precision, and calibration should guide selection. A binary positive-versus-non-positive formulation could be considered for some applications, but only if it matches the operational question.

The second priority is representation quality. Word-level bigrams can preserve phrases such as “not worth” and “book online.” Subword or transformer tokenization would reduce the effect of spelling variants. Sentence-BERT embeddings could be evaluated for classification and clustering, while pretrained transformer classifiers could provide a stronger deep-learning baseline than a randomly initialized three-epoch BiLSTM.

For topic modelling, coherence-driven topic-count selection and human topic labelling should replace the single ten-topic run. Structural topic models or BERTopic could incorporate metadata such as rating, year, season, or review length. Temporal analysis may reveal how queueing, ticketing, or crowding changes over time.

For RAG, the retriever should be evaluated independently before generation. Retrieved evidence should be deduplicated and cited. An instruction-tuned generator should be prompted to answer in short, attributable statements and refuse unsupported questions. Faithfulness checks and a larger evaluation set are essential. The Streamlit app should then load cached artifacts, call the real pipeline, expose sources, collect feedback, and be deployed with a documented URL and reproducible configuration.

12. Conclusion

This project demonstrates a broad NLP pipeline on 8,285 Colosseum visitor reviews. It performs data-quality checks, extensive normalization, sparse vectorization, Word2Vec training, semantic-neighbour search, t-SNE visualization, three sentiment classifiers, SHAP explanation, LDA topic modelling, dense retrieval, language generation, and interface prototyping. The strongest traditional model is XGBoost, with 0.9113 accuracy and 0.47 macro-F1, followed by Logistic Regression at 0.9065 accuracy and 0.41 macro-F1. The BiLSTM reaches majority-class accuracy but only 0.3343 macro-F1.

The main substantive conclusion is that accuracy alone is misleading. Because 90.19% of reviews are Positive, every model appears strong while recognising very few Neutral and Negative instances. XGBoost offers the best recorded minority recall, but its 0.05 Neutral recall remains inadequate. The embedding and topic experiments capture meaningful visitor concepts, including tours, ticketing, history, arena access, and adjacent sites, although punctuation variants and topic overlap limit clarity.

The RAG experiment illustrates why end-to-end evaluation matters. Relevant reviews are retrieved, yet DistilGPT2 produces an unrelated repetitive answer. The interface similarly demonstrates layout but not integration or deployment. These honest negative findings are valuable: they identify the exact transition from prototype components to a dependable application. With imbalance-aware training, improved tokenization, stronger document embeddings, systematic topic selection, grounded generation, and completed deployment, the project could evolve into a useful and trustworthy visitor-review assistant.

References

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research*, *3*, 993-1022.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794).

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735-1780.

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, *33*.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, *30*.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825-2830.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP* (pp. 3982-3992).

Rome Colosseum Visitor Reviews 2019-2026. Kaggle dataset. https://www.kaggle.com/datasets/uradkr/rome-colosseum-visitor-reviews

van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, *9*, 2579-2605.

Appendix A. Recorded Configuration Summary

The principal recorded settings are: an 80:20 stratified split with random state 42; CountVectorizer and TfidfVectorizer with maximum 5,000 features, minimum document frequency five, maximum document frequency 0.8, and English stop words; Word2Vec with 100 dimensions, window five, minimum count five, ten epochs, four workers, and seed 42; t-SNE with 2,000 sampled document vectors, cosine metric, perplexity 30, and random state 42; Logistic Regression with LBFGS and 1,000 maximum iterations; XGBoost with multi-class log loss; BiLSTM with 64-dimensional embeddings, 64 hidden units in each direction, 120-token sequences, Adam learning rate 0.001, batch size 64, and three epochs; LDA with ten topics, two passes, symmetric alpha, automatic eta, and random state 42; and RAG using all-MiniLM-L6-v2, cosine top-five retrieval, and deterministic DistilGPT2 generation up to 120 new tokens.